Sensor Fusion with Biases Estimation using EKF

Prepared by: SEN Kevin

Table of Contents

Sensor Fusion with Biases Estimation using EKF 1

List of Figures 3

1. Introduction 4

2. Theoretical Approach 5

2.1. Translational Dynamics 5

2.2. Translational Dynamics 6

2.3. Translational Kinematics 6

2.4. Rotational Dynamics 6

2.5. Rotational Kinematics 6

3. Methodology 7

3.1. Gyroscope Mathematical Model 7

3.2. Accelerometer Mathematical Model 7

3.3. Extended Kalman Filter Algorithm (EKF) 7

3.4. Simulation 9

4. Results and Discussion 10

5. Conclusion 13

6. References 14

List of Figures

Figure 2.1: The Aerodynamic force D induced by the propeller (Beard & McLain, 2025)..... 5

Figure 3.1: Attitudinal Control Architecture 9

Figure 4.1: Actual Down position vs Desired Down Position..... 10

Figure 4.2: Actual East position vs Desired East Position 10

Figure 4.3: Actual Roll angle $\phi actual$ vs Estimated angle ϕ_{EKF}..... 11

Figure 4.4: Actual Roll angle $\theta actual$ vs Estimated angle θ_{EKF}..... 11

Figure 4.5: Estimated Biases bx & by vs the actual bias..... 12

1. Introduction

Sensor fusion is a critical component of modern aerospace and robotics systems, enabling accurate state estimation by combining complementary data from multiple sensors. While various filtering techniques exist, such as complementary filters and particle filters, the Extended Kalman Filter is selected for this framework due to its strong performance in handling nonlinear system dynamics.

This project implements a complete simulation model in MATLAB and Simulink to evaluate EKF-based attitude and position estimation for an aerial vehicle. Specifically, the simulation incorporates mathematical models for an accelerometer and a gyroscope, explicitly accounting for and estimating time-varying gyro biases alongside vehicle states. The structure of this report details the theoretical modeling of translational and rotational dynamics, the algorithmic implementation of the EKF, simulation architecture, and an analysis of state estimation and bias convergence performance.

Furthermore, this simulation framework serves as a foundational step toward the development and testing of a custom hardware flight controller. By validating the Extended Kalman Filter and bias estimation algorithms in a controlled simulation environment, this project establishes reliable state estimation logic intended for future integration into a physical drone platform.

2. Theoretical Approach

2.1. Translational Dynamics

The translational dynamics of the drone can be derived by using Newton's Second Law of motion $\sum F = ma$ it is the results of the sum of vector of forces acting on the drone or the system expressed in the global frame which includes the aerodynamic forces D, gravitational force $F_{Gravity} = mg$, and the thrust produced by the propellers T.

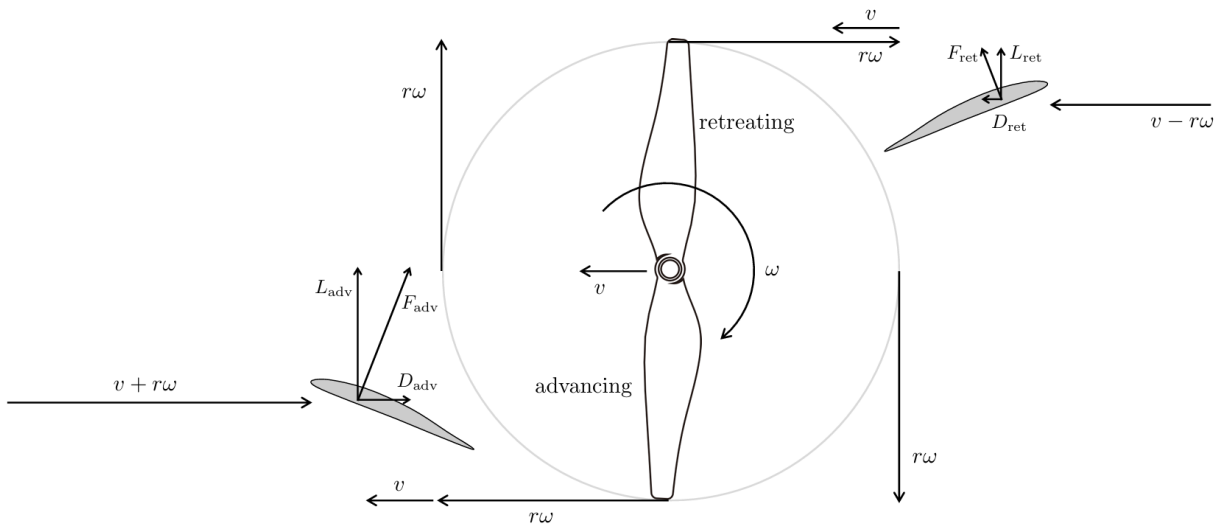


Figure 2.1: The Aerodynamic force D induced by the propeller (Beard & McLain, 2025)

The mathematical expression for the translational Dynamics can be written as:

$$\dot{v}_b^i = ge_3 - R_b^i D(R_b^i)^T v_b^i - \frac{T}{m}(R_b^i)e_3 \qquad \textbf{Eq. 2.1}$$

where

g is the gravitational acceleration ($g = 9.81\ m/s$)

T is the thrust produced by the propellers

$\dot{v}_b^i$ & v_b^i are the body acceleration and velocity respectively

m is the mass of the drone or the system

$D = \begin{pmatrix} gC_d & 0 & 0 \\ 0 & gC_d & 0 \\ 0 & 0 & gC_d \end{pmatrix}$ is the aerodynamic drag of the drone

$e_3 = [1\ 0\ 0]^T$

2.2. Translational Dynamics

In this section, we establish the mathematical expression for the translational dynamics was established for the convenient of building the mathematical modelling of the accelerometer.

The Model of the Translational Dynamics of the Drone can be written as:

$$a_{acc} = \dot{v}_b + (\omega_b \times v_b)$$

2.3. Translational Kinematics

The translational Kinematics can be mathematically expressed as

$$\dot{P}_b^i = v_b^i \qquad \textbf{Eq. 2.2}$$

2.4. Rotational Dynamics

The mathematical expression of the Rotational dynamics can be written as:

$$\dot{\omega}_b = J^{-1}[-\omega_b \times (J\omega_b) + m^b] \qquad \textbf{Eq. 2.3}$$

Where $\omega_b = (p, q, r)^T$ is the angular velocity in the body frame

2.5. Rotational Kinematics

The mathematical expression of the Rotational Kinematics can be written as:

$$\begin{pmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{pmatrix} = \begin{pmatrix} 1 & \sin\phi\tan\theta & \cos\phi\tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi\sec\theta & \cos\phi\sec\theta \end{pmatrix} \begin{pmatrix} p \\ q \\ r \end{pmatrix} \qquad \textbf{Eq. 2.4}$$

3. Methodology

3.1. Gyroscope Mathematical Model

The mathematical model of the gyroscope is really simple as we only need the body frames computed from **Eq. 2.3** above. But in order to push this model to match the physical real world gyroscope as much as possible, the biases in the x and y axes will be added into the body frame angular velocities.

So, the Model of the gyroscope can be written as:

$$p_{gyro} = p_b + b_x \quad \textbf{Eq. 3.1}$$

$$q_{gyro} = q_b + b_y \quad \textbf{Eq. 3.2}$$

Notice that the yaw rate r_{gyro} is not included in the equations above, the yaw rate is not considered in this gyro modelling because in this frame work the we only fuse the gyroscope and the accelerometer together which will be discussed much more thoroughly in later sections.

3.2. Accelerometer Mathematical Model

Accelerometer is one of the most essential part in this sensor fusion frame work because it corrects the time update of the gyro.

The Mathematical model of the Accelerometer can be expressed as:

$$a_{imu} = \dot{v}_b + \begin{pmatrix} qw - rv \\ ru - pw \\ pv - qu \end{pmatrix} + \begin{pmatrix} -g\sin\theta \\ g\sin\phi\cos\theta \\ g\cos\phi\cos\theta \end{pmatrix} \quad \textbf{Eq. 3.3}$$

Where $\dot{v}_b$ is the velocity of the drone expressed in the body frame

$\begin{pmatrix} qw - rv \\ ru - pw \\ pv - qu \end{pmatrix}$ is the coriolis term

$\begin{pmatrix} -g\sin\theta \\ g\sin\phi\cos\theta \\ g\cos\phi\cos\theta \end{pmatrix}$ is the gravitational acceleration term expressed in the body frame.

3.3. Extended Kalman Filter Algorithm (EKF)

Extended Kalman Filter or EKF is a really famous algorithm widely used for sensor fusion, data fusion, and parameters estimation.

Jacobian matrix:

$$A = \begin{bmatrix} \frac{\partial \dot{\phi}}{\partial \phi} & \frac{\partial \dot{\phi}}{\partial \theta} & \frac{\partial \dot{\phi}}{\partial b_x} & \frac{\partial \dot{\phi}}{\partial b_y} \\ \frac{\partial \dot{\theta}}{\partial \phi} & \frac{\partial \dot{\theta}}{\partial \phi} & \frac{\partial \dot{\theta}}{\partial \phi} & \frac{\partial \dot{\theta}}{\partial \phi} \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

where:

$$\frac{\partial \dot{\phi}}{\partial \phi} = \left(q - b_y\right)(cos\,\phi\, tan\,\theta) - r\, sin\,\phi\, tan\,\phi$$

$$\frac{\partial \dot{\phi}}{\partial \theta} = \left(q - b_y\right)\sin\phi\left(\frac{1}{\cos^2\theta}\right) + r\, cos\,\phi\left(\frac{1}{\cos^2\theta}\right)$$

$$\frac{\partial \dot{\phi}}{\partial b_x} = -1$$

$$\frac{\partial \dot{\phi}}{\partial b_y} = -\sin\phi\tan\theta$$

$$\frac{\partial \dot{\theta}}{\partial \phi} = \left(b_y - q\right)\sin\phi - r\cos\phi$$

$$\frac{\partial \dot{\theta}}{\partial \theta} = 0$$

$$\frac{\partial \dot{\theta}}{\partial b_x} = 0$$

$$\frac{\partial \dot{\theta}}{\partial b_y} = -\cos\phi$$

So, the Jacobian Matrix will be rewritten as:

$$A = \begin{bmatrix} \left(q - b_y\right)(cos\,\phi\, tan\,\theta) - r\, sin\,\phi\, tan\,\phi & \left(q - b_y\right)\sin\phi\left(\frac{1}{\cos^2\theta}\right) + r\, cos\,\phi\left(\frac{1}{\cos^2\theta}\right) & -1 & -\sin\phi\tan\theta \\ \left(b_y - q\right)\sin\phi - r\cos\phi & 0 & 0 & -\cos\phi \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Algorithm: Extended Kalman Filter For Parameter Estimation

1. Initialization check: Define initial state estimates $\hat{x}_o = \left[\phi_{est}, \theta_{est}, b_x, b_y\right]^T$, Define initial Covariance P_o, process noise Q_d, measurement noise R

2. Time Update:
3. $x_k = x_{k-1} + dtf(x_{k-1}, u_{k-1})$
4. $A_{k-1} = I + dt\frac{\partial f}{\partial x}|_{x=\hat{x}_{k-1|k-1}}$
5. $P_{k|k-1} = A_{k-1}P_{k-1|k-1}A_{k-1}^T + Q_d$
6. Measurement Update:
7. $W_k = P_{k|k-1}C_k^T\left(C_kP_{k|k-1}C_k^T + R\right)^{-1}$
8. $\hat{y}_{k|k-1} = h_d(\hat{x}_{k|k-1}, u_k)$
9. $\hat{x}_{k|k-1} = W_k\left(y_k - \hat{y}_{k|k-1}\right)$
10. $P_{k|k} = P_{k|k-1} - W_kC_kP_{k|k-1}$

3.4. Simulation

The simulation was conducted by inputting the desired East position of $y = 5\,m$ and the North position of $z = -5\,m$ or $h = 5\,m$ into the position controller or the outer loop to generate the commanded attitude ($\phi_{command}$) and Thrust ($T_{command}$) which are fed into the attitude control but in this attitude controller the actual angles fed to compute the attitudinal error are from the Sensor Fusion block using EKF they are not the angles computed from the rotational dynamics as shown in **Figure 3.1** of the system to generate the commanded torque ($\tau_{command}$) and feed them into the inner loop or the angular rate controller.

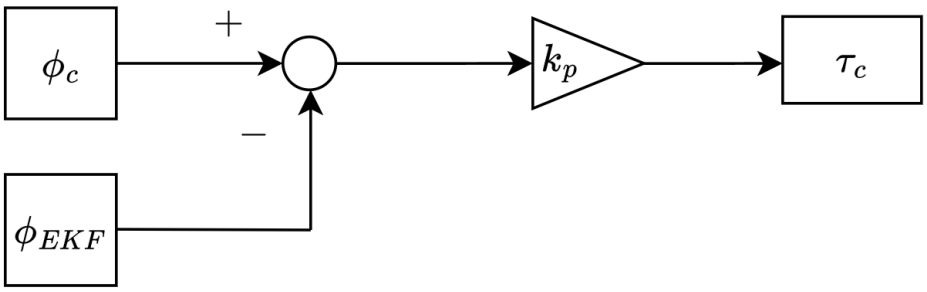


Figure 3.1: Attitudinal Control Architecture

4. Results and Discussion

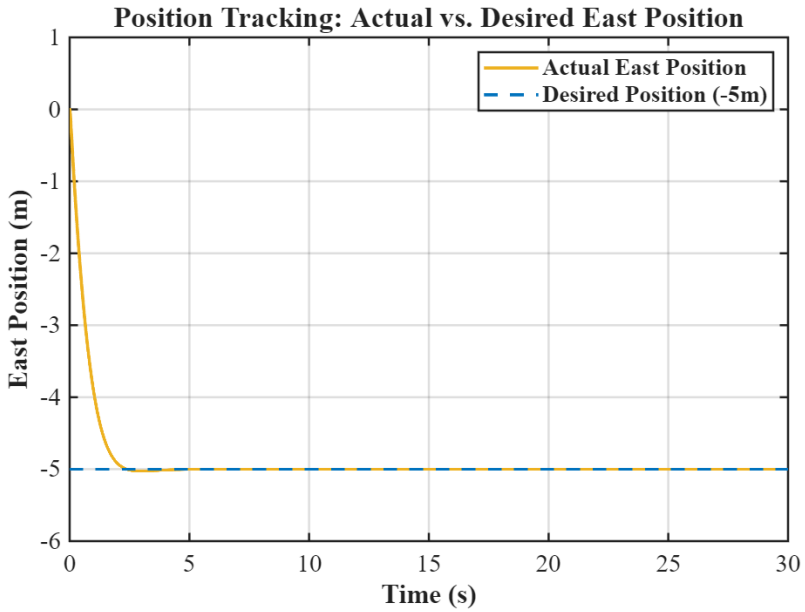


Figure 4.1: Actual Down position vs Desired Down Position

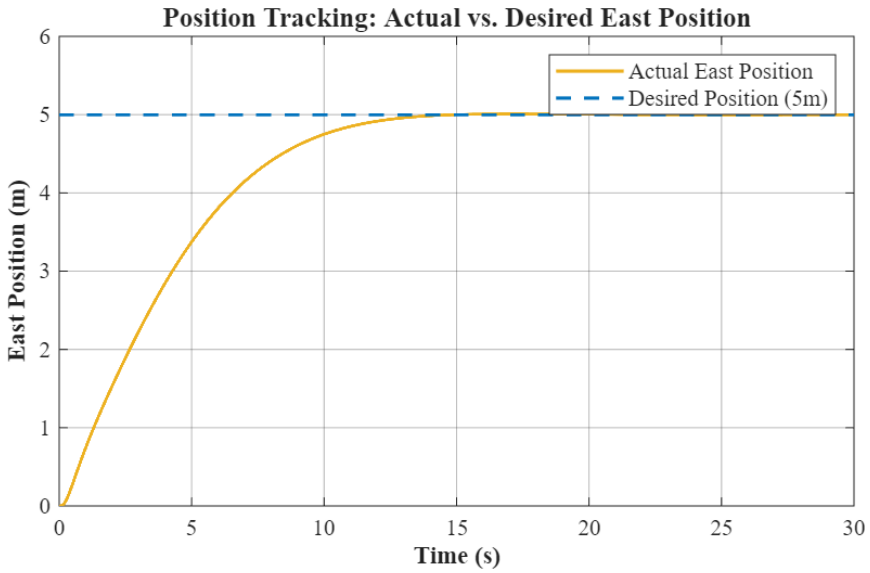


Figure 4.2: Actual East position vs Desired East Position

From **Figure 4.1,** it can be seen that the drone gradually increased its altitude and held the position perfectly when it reached the $-5\ m$ altitude. Just the altitude control, the drone also rolled itself to tracked the desired east position as well it took the drone about $13\ s$ to reach the desired east position of about $5\ m$ consequently, the position controller generated the desired angles as a

reference for the attitude controller where the estimated angles (ϕ_{EKF} & θ_{EKF}) were fed into the controller which we then did the analysis and plot the Estimated Angle (ϕ_{EKF} & θ_{EKF}) and Actual Angles (ϕ_{actual} & θ_{actual}) which are shown in the figures below.

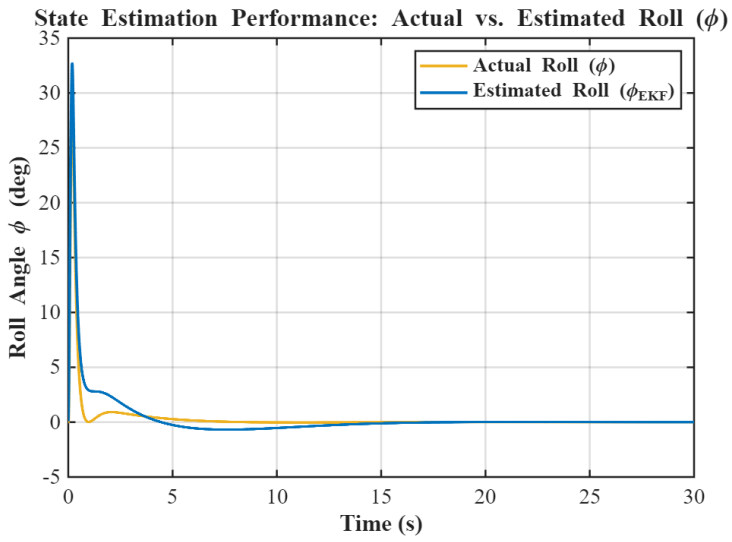


Figure 4.3: Actual Roll angle ϕ_{actual} vs Estimated angle ϕ_{EKF}

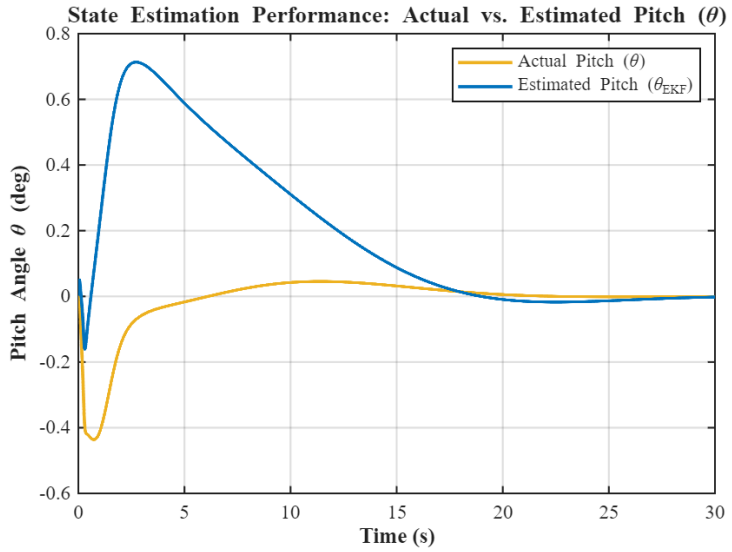


Figure 4.4: Actual Roll angle θ_{actual} vs Estimated angle θ_{EKF}

Figure 4.3 and **Figure 4.4** illustrate the differences between the estimated angles (ϕ_{EKF} & θ_{EKF}) and the actual angles (ϕ_{actual} & θ_{actual}). The overall results show that there are slight differences between the estimated angles and the actual angles during the ascending and maneuvering phase as shown in **Figure 4.3** at $t = 1\ s$, the sudden maneuver and ascent introduced the unaccounted

accelerations which affect our roll angle (ϕ_{EKF}) estimation resulted in the differences of about 3 ° differences between the actual angle and the estimated angle. However, just as soon as the drone reaches the desired altitude at about $t = 3\ s$ the accelerometer is stable enough to give accurate measurements of the angles and starts estimating the gyro bias consequently, the estimated angles starts converging and tracking the actual angles more accurately. Likewise, the estimated pitch angles shown in **Figure 4.4**, also gave similar results to the roll angle estimation during the maneuver and ascent. Though, we would intuitively think that the actual pitch angle which in this case is the yellow curve shown in would be a dead flat line of 0° which is theoretically accurate under ideal conditions where the sensor tracks the actual angle perfectly but since the actual angles fed into the attitudinal controller in this simulation and frame work were not the angles computed from the dynamics equation rather is the estimated angles from the EKF sensor fusion and since the gyro scope also includes the bias of $1°/s$ as a results these biases could affect the time update to give inaccurate results for about 0.7 ° which is then fed into the controller. The controller misinterpreted this as an error because we did not set the desired North position for the position controller hence, the Commanded pitch angle output from the position controller would be $\theta_{command} = 0°$ but during the ascent and maneuver the EKF did not estimate the accurate results because of the slow convergence of the biases estimation as shown in **Figure 4.5** below so the controller would try to flatten it out which further effect the actual angle but this error is too small to effect the overall performance of the controller. Because physically speaking an error of 0.7° is barely visible to the naked human eyes.

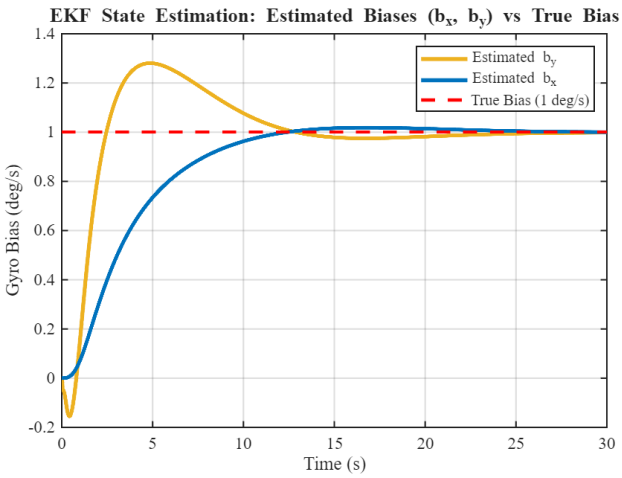


Figure 4.5: Estimated Biases b_x & b_y vs the actual bias

5. Conclusion

The Extended Kalman Filter (EKF) successfully estimated and tracked both roll and pitch angles during the UAV simulation. While sudden maneuvers and the initial ascent introduced temporary discrepancies due to the accelerations which were not accounted and the slow gyro bias convergence, these differences between the actual angles and the estimated angles were short lived. Once the vehicle reached its desired position and the sensor readings stabilized, the EKF rapidly converged to accurately track the actual states. Although minor transient discrepancies propagated through the controller loop, their overall magnitude was negligible and had no adverse effect on the overall flight controller performance.

6. References

Beard, R. W., & McLain, T. W. (2025). *Small unmanned aircraft: theory and practice.* Princeton University Press.